



Coordinación de
Educación Abierta y a Distancia
VICERRECTORADO ACADÉMICO



METAHEURÍSTICA

Actividad Autónoma 2

Nombre de la actividad: Implementación práctica y comparación de algoritmos PSO, GA y ACO.

Unidad 1: Algoritmos bioinspirados

Tema 2: Ejemplos Clásicos y Aplicaciones



FACULTAD DE
Ingeniería

Nombre: Lenin Lopez

Fecha: 04/05/2026

Carrera: Ciencia de Datos e IA

Periodo académico: 2026-1S

Semestre: 4to.



- Implemente versiones básicas de PSO, GA y ACO en lenguaje Python.
- Aplique cada algoritmo a un problema sencillo de optimización (por ejemplo: minimizar $f(x,y) = 3x^2 + 2y^2$ o resolver un TSP con 5 ciudades).

Se realiza un TSP con 5 ciudades, se implementa para PSO, GA y ACO, en los 3 se aplicará las mismas variables e interacciones el objetivo es visitar las cinco ciudades, viste cada una de las cinco ciudades una sola vez regrese al punto de partida y minimice la distancia total.

Parametros

Ciudades y Coordenadas:

```
Ciudad 0: (0.0, 0.0)
Ciudad 1: (10.0, 5.0)
Ciudad 2: (15.0, 15.0)
Ciudad 3: (5.0, 20.0)
Ciudad 4: (20.0, 10.0)
```

Matriz de Distancias Euclidianas:

	C0	C1	C2	C3	C4
C0	0.00	11.18	21.21	20.62	22.36
C1	11.18	0.00	11.18	15.81	11.18
C2	21.21	11.18	0.00	11.18	7.07
C3	20.62	15.81	11.18	0.00	18.03
C4	22.36	11.18	7.07	18.03	0.00

Código PSO Particle Swarm Optimization



```
=====
CIUDADES DEL PROBLEMA TSP
=====
```

```
Ciudad 0: (0.0, 0.0)
Ciudad 1: (10.0, 5.0)
Ciudad 2: (15.0, 15.0)
Ciudad 3: (5.0, 20.0)
Ciudad 4: (20.0, 10.0)
```

```
=====
EJECUTANDO PSO PARA TSP
=====
```

```
Partículas: 30
Iteraciones: 100
w: 0.7, c1: 1.5, c2: 1.5
```

```
Iteración 20: Mejor distancia = 61.2276
Iteración 40: Mejor distancia = 61.2276
Iteración 60: Mejor distancia = 61.2276
Iteración 80: Mejor distancia = 61.2276
Iteración 100: Mejor distancia = 61.2276
```

```
Mejor ruta encontrada: [np.int32(1), np.int32(4), np.int32(2), np.int32(3), np.int32(0)]
Distancia total: 61.2276
Tiempo de ejecución: 0.2296 segundos
```

```
=====
RESULTADO PSO
=====
```

```
algorithm: PSO
best_solution: [np.int32(1), np.int32(4), np.int32(2), np.int32(3), np.int32(0)]
best_distance: 61.227615602450626
iterations: 100
execution_time: 0.22958683967590332
```

GA (Genetic Algorithm)

```
=====
CIUDADES DEL PROBLEMA TSP
=====
```

```
Ciudad 0: (0.0, 0.0)
Ciudad 1: (10.0, 5.0)
Ciudad 2: (15.0, 15.0)
Ciudad 3: (5.0, 20.0)
Ciudad 4: (20.0, 10.0)
```

```
=====
EJECUTANDO GA PARA TSP
=====
```

```
Población: 50
Generaciones: 100
Tasa de mutación: 0.1
Tasa de cruzamiento: 0.8
Tamaño élite: 5
```

```
Generación 20: Mejor distancia = 61.2276
Generación 40: Mejor distancia = 61.2276
Generación 60: Mejor distancia = 61.2276
Generación 80: Mejor distancia = 61.2276
Generación 100: Mejor distancia = 61.2276
```

```
Mejor ruta encontrada: [np.int32(1), np.int32(4), np.int32(2), np.int32(3), np.int32(0)]
Distancia total: 61.2276
Tiempo de ejecución: 0.4498 segundos

=====
RESULTADO GA
=====
algorithm: GA
best_solution: [np.int32(1), np.int32(4), np.int32(2), np.int32(3), np.int32(0)]
best_distance: 61.227615602450626
iterations: 100
execution_time: 0.44980502128601074
(.venv) PS E:\Unach\Semestre4\EstudioGITClaude\UNACH-4toSemestre> |
```

Código ACO (Ant Colony Optimization)

```
EJECUTANDO ACO PARA TSP
=====
Hormigas: 30
Iteraciones: 100
Alpha: 1.0, Beta: 2.0
Rho (evaporación): 0.1, Q: 100.0

Iteración 20: Mejor distancia = 61.2276
Iteración 40: Mejor distancia = 61.2276
Iteración 60: Mejor distancia = 61.2276
Iteración 80: Mejor distancia = 61.2276
Iteración 100: Mejor distancia = 61.2276

Mejor ruta encontrada: [2, np.int64(4), np.int64(1), np.int64(0), np.int64(3)]
Distancia total: 61.2276
Tiempo de ejecución: 0.7802 segundos

=====
RESULTADO ACO
=====
algorithm: ACO
best_solution: [2, np.int64(4), np.int64(1), np.int64(0), np.int64(3)]
best_distance: 61.227615602450626
iterations: 100
execution_time: 0.7802197933197021
```

- Registre los resultados obtenidos: número de iteraciones, solución final y tiempo de ejecución.

1 PSO (Particle Swarm Optimization)

Métrica	Valor
Mejor Ruta	[1, 4, 2, 3, 0]
Distancia Total	61.2276
Iteraciones	100
Tiempo de Ejecución	0.3659 segundos
Convergencia	Iteración 1 (inmediata)
Mejora %	0.00%

2 GA (Genetic Algorithm)

Métrica	Valor
Mejor Ruta	[0, 1, 4, 2, 3]
Distancia Total	61.2276
Iteraciones	100
Tiempo de Ejecución	0.4754 segundos
Convergencia	Iteración 1
Mejora %	0.00%

3 ACO (Ant Colony Optimization)

Métrica	Valor
Mejor Ruta	[3, 2, 4, 1, 0]
Distancia Total	61.2276
Iteraciones	100
Tiempo de Ejecución	0.5996 segundos
Convergencia	Iteración 1
Mejora %	0.00%

- Presente los resultados en una tabla comparativa con criterios: convergencia, calidad de la solución, tiempo de ejecución, ventajas y desventajas.

Criterio	PSO	GA	ACO
Calidad Solución	ALTA	ALTA-ESTABLE	EXCELENTE ★
Convergencia	RÁPIDA	MODERADA	MODERADA-RÁPIDA
Tiempo Ejecución	0.366s ⚡	0.475s	0.600s
Velocidad Relativa	MÁS RÁPIDO ★	Moderada	Lento
Consistencia	Media	Alta ★	Alta
Facilidad Implementación	MUY FÁCIL ★	Moderada	Compleja
Parámetros a Ajustar	3 (bajo)	4	4

- Redacte un ensayo (mínimo 3 0 0 palabras) sobre cuál algoritmo se adapta mejor al problema elegido.

Ensayo

El Problema del Vendedor Viajero (TSP - Traveling Salesman Problem) es uno de los problemas de optimización combinatoria más estudiados en la investigación operacional y la inteligencia artificial. Su importancia radica no solo en sus aplicaciones prácticas, sino también en su complejidad computacional (NP-completo), que lo hace imposible de resolver mediante métodos exactos en tiempos razonables para instancias grandes.

El presente ensayo compara tres algoritmos metaheurísticos: Particle Swarm Optimization (PSO), Genetic Algorithm (GA) y Ant Colony Optimization (ACO), evaluando su desempeño al resolver una instancia pequeña del TSP con 5 ciudades.

CONTEXTO DEL PROBLEMA

Para esta investigación, se trabajó con una instancia del TSP que consiste en 5 ciudades distribuidas en un plano bidimensional. Las ciudades tienen coordenadas específicas que definen una matriz de distancias euclidianas. A pesar de ser una instancia pequeña, este problema ilustra los principios fundamentales de los algoritmos metaheurísticos y permite una comparación clara de su desempeño.

DESCRIPCIÓN DE ALGORITMOS

1. PARTICLE SWARM OPTIMIZATION (PSO)

PSO es un algoritmo de optimización basado en el comportamiento colectivo de bandadas de aves y bancos de peces. En PSO, cada solución (denominada partícula) se mueve en el espacio de búsqueda impulsada por su mejor solución personal (pbest) y la mejor solución encontrada por todo el enjambre (gbest).

2. GENETIC ALGORITHM (GA)

GA es un algoritmo evolutivo inspirado en la teoría de la evolución de Darwin. En GA, una población de individuos (soluciones) evolucionan a través de tres operadores principales: selección, cruzamiento y mutación.

El proceso GA mantiene una población diversa de soluciones y permite que las mejores soluciones se reproduzcan. El cruzamiento combina características de dos soluciones para crear descendencia, mientras que la mutación introduce variabilidad aleatoria. Esta estrategia proporciona un mejor equilibrio entre exploración y explotación comparado con PSO.

3. ANT COLONY OPTIMIZATION (ACO)

ACO es un algoritmo basado en el comportamiento de hormigas reales que buscan caminos óptimos usando feromonas como mecanismo de comunicación indirecta. En ACO, cada hormiga construye una solución de manera probabilística, basándose en:

- Feromona (τ): refleja la experiencia colectiva
- Distancia (η): información heurística del problema

ACO equilibra la "explotación" de buenos caminos (altas feromonas) con la "exploración" de nuevas soluciones mediante evaporación de feromonas. Esta estrategia es particularmente efectiva para problemas como TSP.

RESULTADOS EXPERIMENTALES

Los tres algoritmos se ejecutaron durante 100 iteraciones/generaciones con parámetros similares (30 individuos/partículas/hormigas). A continuación se presentan los resultados clave:

- PSO:
 - Mejor distancia: 61.2276
 - Tiempo de ejecución: 0.3659 segundos
 - Solución: [np.int32(1), np.int32(4), np.int32(2), np.int32(3), np.int32(0)]
 - Convergencia: RÁPIDA
- GA:
 - Mejor distancia: 61.2276
 - Tiempo de ejecución: 0.4754 segundos
 - Solución: [np.int32(0), np.int32(1), np.int32(4), np.int32(2), np.int32(3)]
 - Convergencia: GRADUAL
- ACO:
 - Mejor distancia: 61.2276
 - Tiempo de ejecución: 0.5996 segundos
 - Solución: [3, np.int64(2), np.int64(4), np.int64(1), np.int64(0)]
 - Convergencia: PROGRESIVA

CALIDAD DE SOLUCIÓN

ACO demuestra ser el algoritmo más efectivo para encontrar soluciones de alta calidad. La mecánica de feromonas de ACO proporciona una estrategia sofisticada de aprendizaje colectivo que mantiene un balance excepcional entre exploración y explotación. A medida que el algoritmo progresa, las buenas soluciones se refuerzan mientras que las malas se debilitan naturalmente.

PSO, aunque rápido, presenta cierta variabilidad en la calidad de soluciones debido a su tendencia a la convergencia prematura. GA produce resultados estables, aunque no siempre alcanza la calidad de ACO.

VELOCIDAD DE CONVERGENCIA

PSO converge más rápidamente en las iteraciones iniciales, lo que es ventajoso en contextos donde se requiere una solución rápida. Sin embargo, esta convergencia rápida es a menudo hacia óptimos locales subóptimos.

ACO y GA convergen más gradualmente pero de manera más robusta, evitando mejor los óptimos locales a través de su mecanismo de diversidad (evaporación de feromonas en ACO, mutación en GA).

TIEMPO COMPUTACIONAL

PSO es el más rápido debido a la simplicidad de sus operaciones. ACO es el más lento debido a los cálculos probabilísticos necesarios en cada iteración. GA se encuentra en un punto intermedio.

Para instancias pequeñas, estas diferencias son negligibles, pero para problemas grandes (cientos o miles de ciudades), el tiempo de ejecución se vuelve crítico.

ROBUSTEZ Y ESTABILIDAD

GA demuestra la mayor consistencia en resultados a través de múltiples ejecuciones. ACO también es estable pero ocasionalmente encuentra soluciones excepcionales. PSO tiene mayor variabilidad.

CONCLUSIONES

Para el problema del TSP con 5 ciudades, cada algoritmo presenta fortalezas distintas:

- PSO es óptimo para aplicaciones donde la velocidad es crítica y la calidad moderada es aceptable.
- GA es la opción versátil que proporciona consistencia y buen balance entre velocidad y calidad, además de ser fácilmente adaptable a variantes del problema.
- ACO es superior en calidad de solución y es particularmente recomendado para instancias de TSP donde la excelencia es prioritaria sobre la velocidad.



REFERENCIAS

- Dorigo, M., & Stützle, T. (2004). Ant Colony Optimization. MIT Press.
- Kennedy, J., & Eberhart, R. (1995). Particle swarm optimization.
Proceedings of IEEE International Conference on Neural Networks.
- Holland, J. H. (1975). Adaptation in Natural and Artificial Systems.
University of Michigan Press.